# Foedrum

Your issues brewed into PRs

## From issue to pull request

Meet the brewers, in the order they work.



Every issue is an order at the Foedrum brewery.
A crew of otters brews it into a pull request, one step at a time.

github.com/EdwinBekaert/foedrum

CHAPTER 1

# An order comes in

An issue walks into the taproom like a guest at the inn. Before anything is brewed, it gets the right recipe, and the brewers get everything they need to know.






**Picks the recipe.** Reads each new order and sends it the right way: Nip for a quick fix, Pint for a feature, Infection check for a bug. One look, one decision.



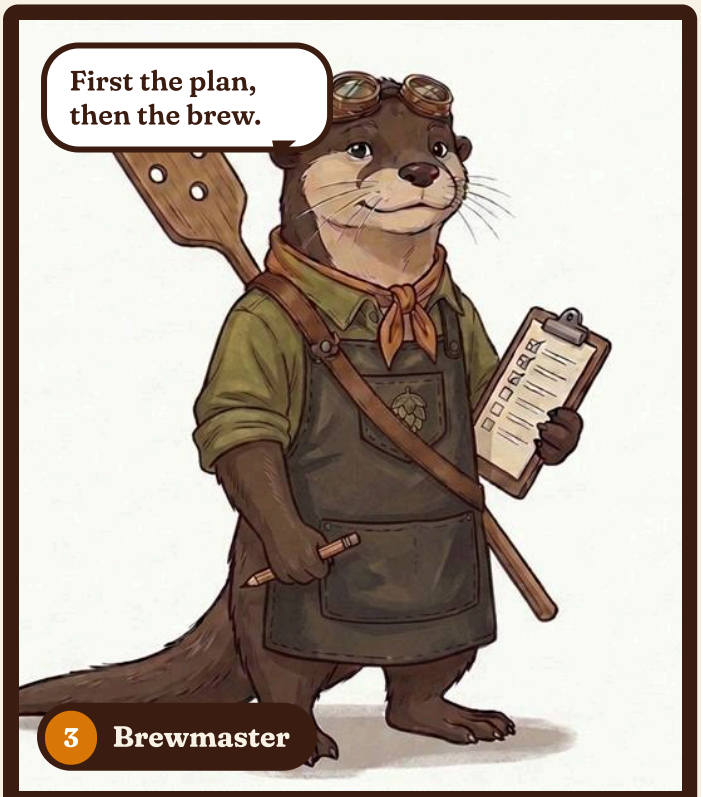

**Gathers the context.** Collects the relevant code, its history and the constraints, so nobody brews blind.

CHAPTER 2

# The brewhouse

With a plan on the clipboard, the batch is brewed test-first: first measure what "done" means, then brew until it measures right.






**Writes the plan.** Turns the order into a plan with acceptance criteria: what done looks like, ticked off one by one.



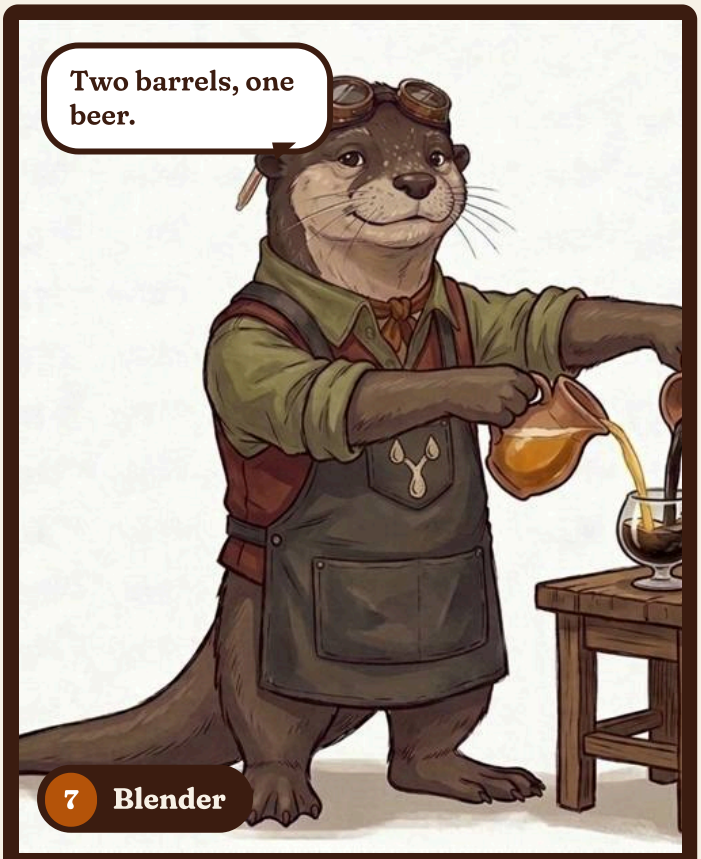

**Writes the failing tests first.** Tests and minimal stubs, no implementation. Foedrum then runs them and checks they really fail: the *red proof*.




**Makes the tests pass.** Writes the code until the Gauger's tests turn green, without changing a single test. Also writes the pull request's text.




**Simplifies the change.** Clears out what isn't needed without changing what the code does, like fining a beer until it's crystal clear.




**Resolves merge conflicts.** The latest main branch is merged into the batch. When the two clash, the Blender blends them into one.

## Gravity reading

No otter here, just measuring. Foedrum runs the repository's own checks on the batch:

`build` `typecheck` `lint` `format` `test`

It costs no model tokens. If a check fails, the batch goes back to the Cellarman.

CHAPTER 3

# Tasting

Nothing leaves the brewery untasted. Several Tasters and Brett go over the batch side by side, each looking for something different.






**Reviews the change.** Each Taster looks through one lens: correctness, acceptance, security, test design. Read-only: tastes, never touches.




**Tests the tests.** Plants plausible bugs in the code; Foedrum checks which ones the tests fail to catch. Named after a wild yeast.




**Fixes the findings.** Repairs what the Tasters found, then the checks run again and the batch is tasted again, up to two rounds.

### The pull request

When the tasting is done, Foedrum opens the pull request with the plan, the tests and the change.

It opens as a **draft** unless every Taster approved. You always have the last word: you merge.

CHAPTER 4

# Served




**Front of house, again.** Shows you what the change lets you do now, and takes your feedback back to the brewery.

CHAPTER 5

# When it's a bug

A bug report gets the Infection check recipe. Instead of the Gauger, the Microbiologist writes the failing test: the one that shows the bug. The rest of the brew is the same.




**Triages the bug.** Reproduces it, finds the root cause, and writes the failing test that proves it.




**Verifies the fix for real.** Tries it in a real environment, for example by clicking through the app in a browser and attaching screenshots.

BACKSTAGE

# Keeping the crew sharp




**Runs the evaluations.** Tastes a flight of example orders to judge how well the brewers and their models do, so the recipes keep improving.

THE WHOLE CREW

# Who's who



**1 · Innkeeper**
picks the recipe



**2 · Maltster**
gathers context



**3 · Brewmaster**
writes the plan



**4 · Gauger**
writes failing tests



**5 · Cellarman**
makes them pass



**6 · Finer**
simplifies the change



**7 · Blender**
resolves merge conflicts



**8 · Taster**
reviews through one lens



**9 · Brett**
plants bugs to test the tests



**10 · Cooper**
fixes the findings



**11 · Tapster**
serves it and takes feedback



**Microbiologist**
triages bugs



**Lab tech**
verifies fixes for real



**Tasting panel**
runs evaluations



**Foedrum**

Your issues brewed into PRs

github.com/EdwinBekaert/foedrum